

Final E2E Report (Evidence-Based) Real-Time Fraud Detection ML System

Repository: Final_Project_DMM501_Group1

2026-04-14

Contents

1	Evidence Pack (What was extracted)	2
1.1	Dataset evidence (Kaggle credit card fraud)	2
1.2	Model artifacts (deployable model + metadata)	2
1.3	Evaluation and explainability figures	2
1.4	Frontend evidence	2
1.5	API evidence	3
1.6	Monitoring and deployment evidence	3
2	Problem Definition	3
2.1	Problem statement	3
2.2	Business context	3
3	Dataset Analysis (with imbalance explanation)	3
3.1	Schema (evidence)	3
3.2	Imbalance (evidence)	3
3.3	Data quality notes (evidence)	4
3.4	Embedded dataset figures	4
4	ML Pipeline	7
4.1	Primary pipeline	7
4.2	Split (evidence)	8
5	Model Training (baseline vs improved)	8
5.1	Models implemented	8
5.2	Best LightGBM hyperparameters (evidence)	8
5.3	Metrics table (evidence)	8
5.4	Additional test-set evidence (accuracy and supports)	8
5.5	Model selection (deployable choice)	9
5.6	Model artifact validation checks (evidence)	9
5.7	Embedded evaluation figures	9
5.8	Final deployable model diagnostics	12

6	Threshold Tuning	15
6.1	How it works (evidence)	15
6.2	Numeric tuning evidence (from validation sweeps)	15
6.3	Embedded threshold figures	16
7	Model Explainability (SHAP)	17
7.1	What is implemented (evidence)	17
8	System Architecture (E2E flow)	18
8.1	Architecture diagram (embedded)	18
8.2	Contract boundary (evidence)	19
9	API Design	19
9.1	Endpoints (implemented)	19
9.2	Schema evidence	19
9.3	Runtime examples	19
10	Frontend System (UI/UX)	19
10.1	Dashboard layout (evidence)	19
10.2	Streaming and alerting logic (evidence)	20
10.3	Embedded frontend screenshot	20
11	API Behavior Examples (Captured Outputs)	21
11.1	How these examples were captured	21
11.2	Example: GET /health	21
11.3	Example: GET /dataset/samples?n=1&strategy=legit	22
11.4	Example: POST /predict	24
12	Monitoring & Alerts	25
12.1	Metrics emitted (evidence)	25
12.2	Prometheus scrape and rules (evidence)	25
12.3	Grafana dashboard (evidence)	25
13	Deployment	27
13.1	Docker Compose stack (evidence)	27
13.2	Deployment contract (model + threshold)	29
14	Testing & CI/CD	29
14.1	Tests (evidence)	29
14.2	GitHub Actions workflows (evidence)	29
15	Responsible AI	29
16	Limitations	29
16.1	Verified vs unverified in this report	29
16.2	Multiple artifact sets (do not conflate)	30
17	Conclusion	30
17.1	End-to-end completeness (implementation)	30
17.2	Demo readiness (next evidence to capture)	30

1 Evidence Pack (What was extracted)

All metrics and file references are from this repository and the generated outputs under `artifacts/`. If a component was not executed live during report generation, it is marked **Unverified**.

1.1 Dataset evidence (Kaggle credit card fraud)

- Dataset file in repo: `data/archive/creditcard.csv`
- Schema validation: `artifacts/reports/dataset_schema.json`
- EDA summary: `artifacts/reports/eda_summary.json`
- Class imbalance: `artifacts/reports/class_distribution.json`
- EDA figures: `artifacts/figures/*.png`

1.2 Model artifacts (deployable model + metadata)

- Deployable model: `artifacts/models/final_model.joblib`
- Deploy metadata: `artifacts/models/model_info.json`
- Model selection summary: `artifacts/reports/model_selection_summary.json`
- Model validation checks: `artifacts/reports/model_validation_checks.json`

1.3 Evaluation and explainability figures

- ROC/PR curves: `artifacts/figures/baseline_*`, `artifacts/figures/improved_*`
- Threshold tuning: `artifacts/figures/*threshold*`, `artifacts/benchmarks/*threshold*`
- Model comparison: `artifacts/figures/model_comparison.png`
- SHAP: `artifacts/figures/shap_summary.png`, `artifacts/benchmarks/shap_importance_table.csv`

1.4 Frontend evidence

- UI layout: `frontend/index.html`
- Streaming logic + risk classification: `frontend/app.js`
- API client (timeouts + schema assertions): `frontend/api-client.js`
- Transaction sources: `frontend/demo-data.js`, `frontend/real-samples.json`
- UI screenshot (layout evidence): `artifacts/figures/frontend_dashboard_live.png`

1.5 API evidence

- FastAPI app: `src/api/main.py`
- Schemas: `src/api/schemas.py`
- Model loader: `src/models/loader.py`
- Dataset sampling: `src/data/samples.py`
- Metrics: `src/monitoring/metrics.py`
- Verification harness: `verify_system.py` (**Unverified** run in this report)

1.6 Monitoring and deployment evidence

- Compose stack: `deployment/docker-compose.yml`
- Prometheus: `deployment/prometheus/prometheus.yml`, `deployment/prometheus/alerts.yml`
- Grafana: `deployment/grafana/dashboards/fraud_api.json` + provisioning
- CI/CD: `.github/workflows/ci.yml`, `.github/workflows/docker.yml`

2 Problem Definition

2.1 Problem statement

Build an end-to-end ML system that assigns a fraud risk score to credit-card transactions and serves predictions via an API, with a demo dashboard for near-real-time monitoring and an observability stack (metrics, dashboards, alerting).

2.2 Business context

Fraud scoring reduces financial loss and manual review cost by prioritizing transactions for investigation. In this system, the decision threshold is treated as an explicit operational policy (stored as model metadata and returned in API responses).

3 Dataset Analysis (with imbalance explanation)

3.1 Schema (evidence)

From `artifacts/reports/dataset_schema.json`: the dataset has 284,807 rows and 31 columns: Time, V1..V28, Amount, Class (`matches_expected_schema=true`).

3.2 Imbalance (evidence)

From `artifacts/reports/class_distribution.json`:

- Class 0 (non-fraud): 284,315 (ratio=0.9982725144; $\approx 99.8273\%$)
- Class 1 (fraud): 492 (ratio=0.0017274856; $\approx 0.1727\%$)

Why accuracy is misleading: With a 0.17% fraud rate, a naive model predicting “non-fraud” for every transaction achieves $\approx 99.83\%$ accuracy. Therefore, this project emphasizes PR-AUC and precision/recall at an operating threshold.

3.3 Data quality notes (evidence)

From `artifacts/reports/eda_summary.json`: duplicate rows = 1081. Missing value counts are exported to `artifacts/reports/missing_values.csv`. Figures 1, 2, and 3 summarize the imbalance and transaction distributions used as input assumptions for model and threshold selection.

3.4 Embedded dataset figures

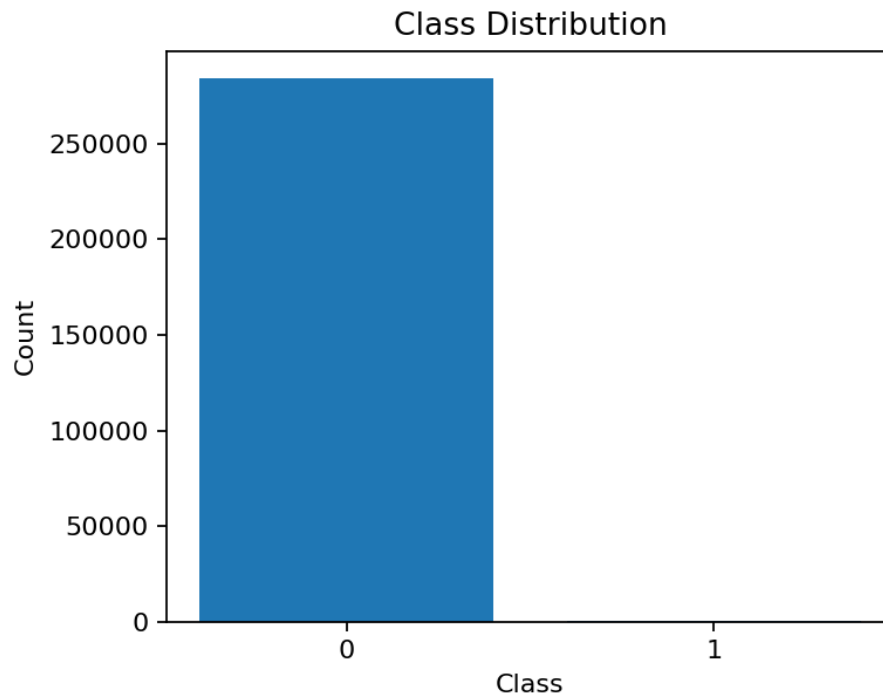


Figure 1: Class distribution (Fraud vs Non-fraud). Source: `artifacts/figures/class_distribution.png`

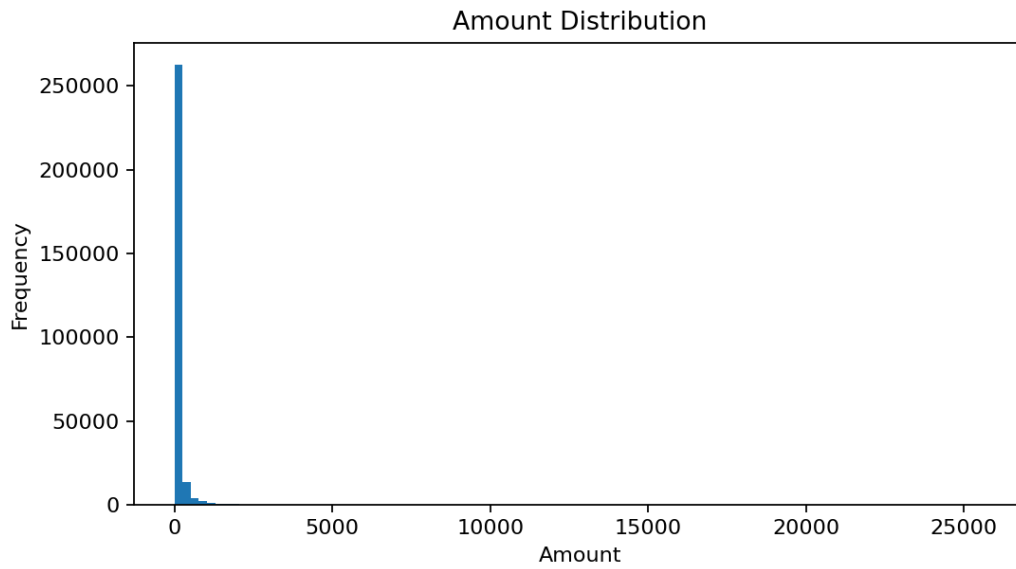


Figure 2: Amount distribution. Source: `artifacts/figures/amount_distribution.png`

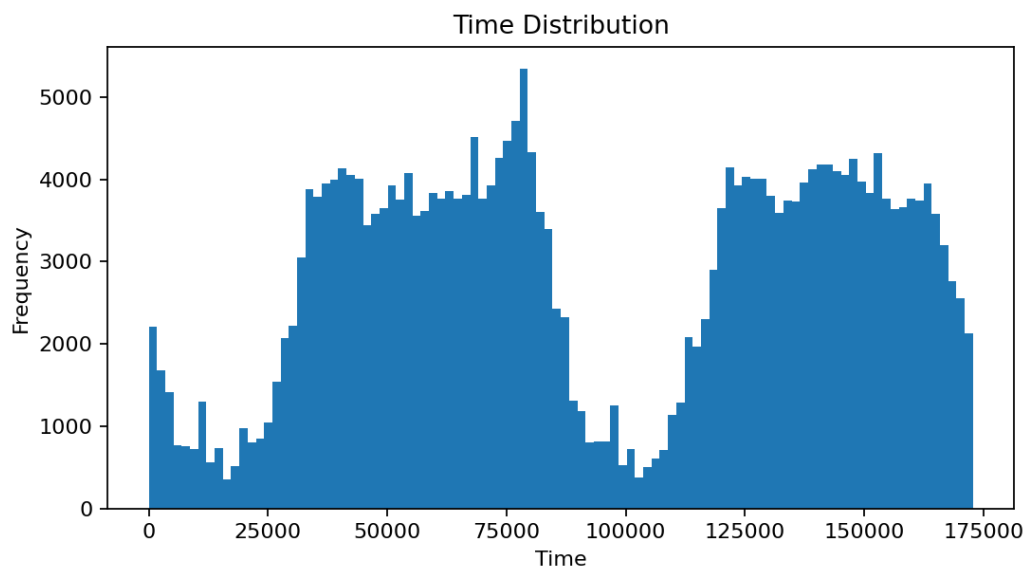


Figure 3: Time distribution. Source: `artifacts/figures/time_distribution.png`

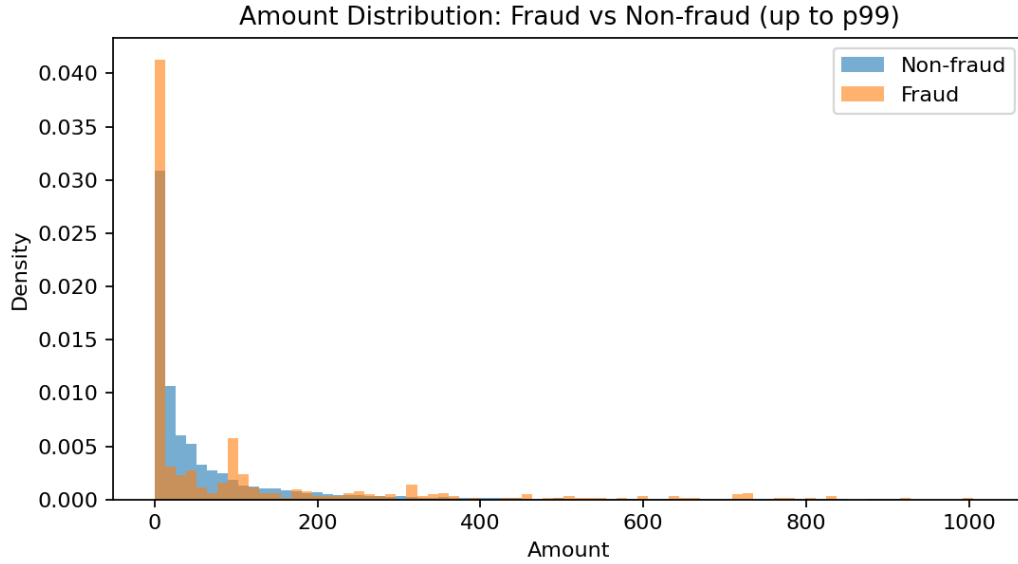


Figure 4: Fraud vs non-fraud amount patterns. Source: `artifacts/figures/fraud_vs_nonfraud_amount.png`

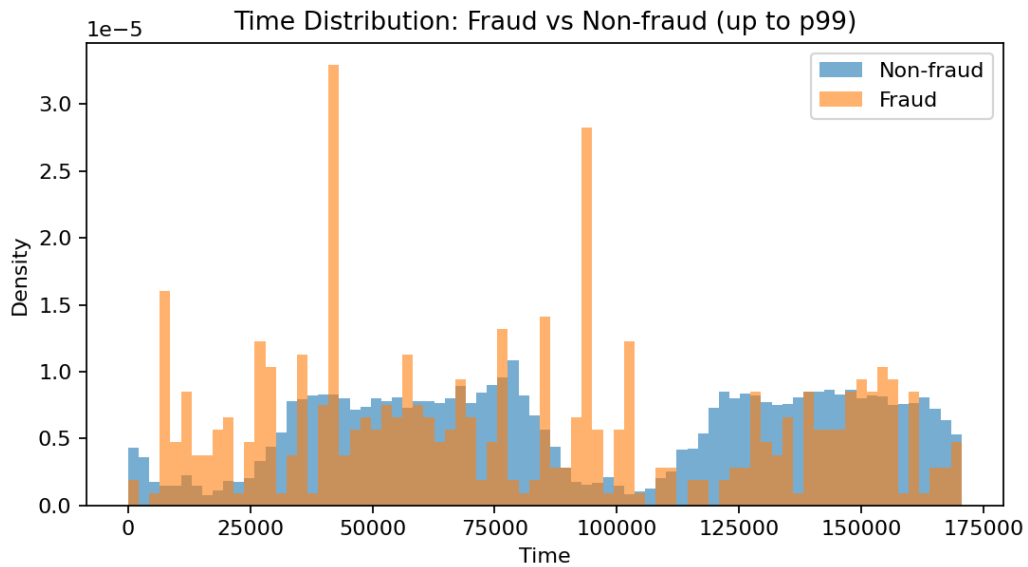


Figure 5: Fraud vs non-fraud time patterns. Source: `artifacts/figures/fraud_vs_nonfraud_time.png`

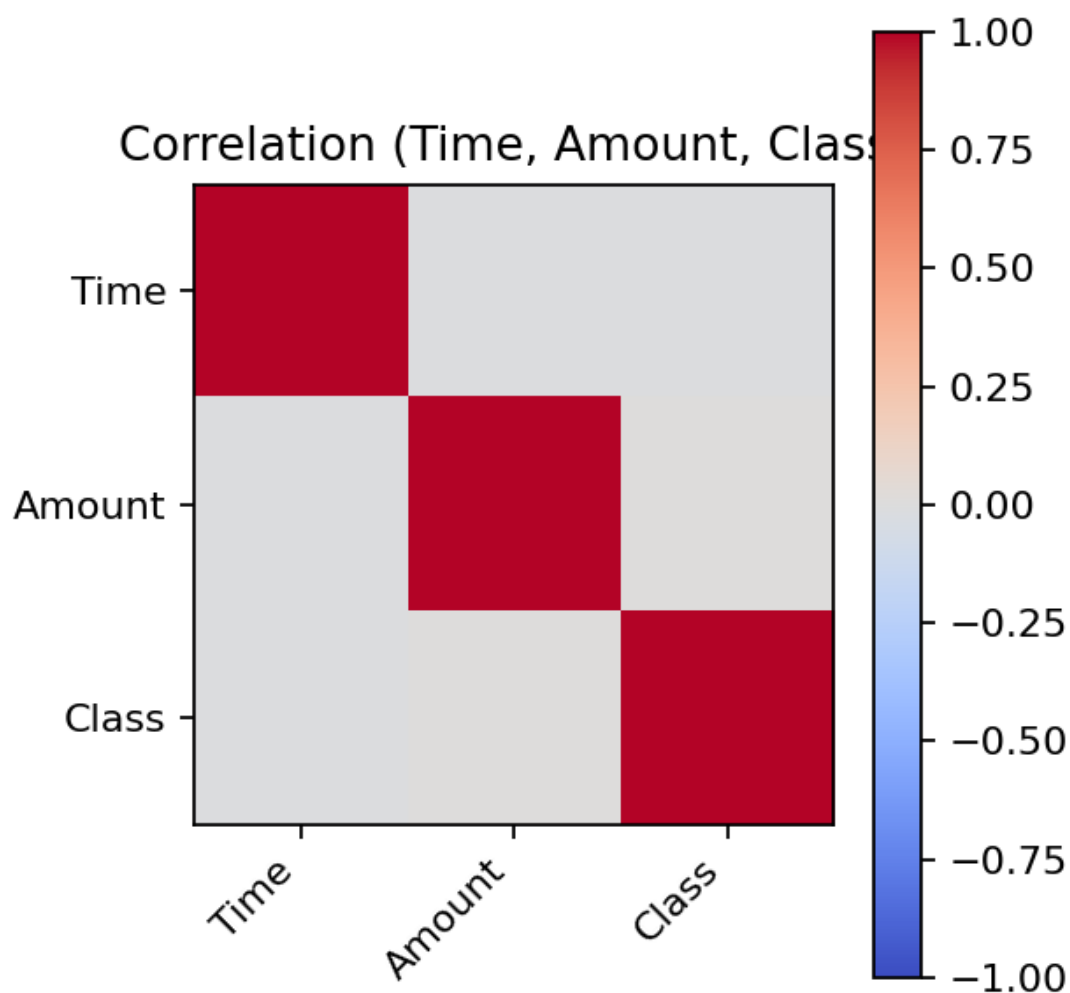


Figure 6: Feature correlation overview. Source: `artifacts/figures/correlation_overview.png`

4 ML Pipeline

4.1 Primary pipeline

The primary benchmark pipeline is implemented in `src/pipelines/run_model_workflow.py`. It performs:

- dataset schema validation and EDA exports
- train/val/test split (seeded)
- baseline training (Logistic Regression pipeline)
- improved training (LightGBM with small hyperparameter sweep)
- threshold tuning (validation sweep; best by max F1)

- evaluation exports (ROC/PR, confusion matrices, classification reports)
- SHAP explainability exports for LightGBM
- model selection (primary criterion PR-AUC) and deployable artifact export

4.2 Split (evidence)

From `artifacts/reports/split_info.json`: train=199,364; val=42,721; test=42,722; n.features=30; seed=42.

5 Model Training (baseline vs improved)

5.1 Models implemented

- Baseline: `scikit-learn Pipeline(StandardScaler, LogisticRegression)` with `class_weight="balanced"`.
- Improved: `lightgbm.LGBMClassifier` tuned by validation PR-AUC across a small `ParameterGrid`. Hyperparameter results: `artifacts/benchmarks/improved_hyperparameter_tuning.csv`.

5.2 Best LightGBM hyperparameters (evidence)

From the top row of `artifacts/benchmarks/improved_hyperparameter_tuning.csv` (sorted by `val_pr_auc`):

- `val_pr_auc`=0.62893
- `learning_rate`=0.03, `n_estimators`=450, `num_leaves`=63
- `scale_pos_weight`=10.0 (natural ratio recorded as ≈ 578.55)

5.3 Metrics table (evidence)

From `artifacts/reports/model_selection_summary.json`:

Model	Thr	PR-AUC	ROC-AUC	Prec	Rec	F1	TN	FP	FN	TP
Logistic Regression (baseline)	0.99	0.7929	0.9680	0.6742	0.8108	0.7362	42619	29	14	6
LightGBM (improved)	0.84	0.7161	0.8828	0.8852	0.7297	0.8000	42641	7	20	5

Table 1: Tuned test metrics (evidence). Source: `artifacts/reports/model_selection_summary.json`

5.4 Additional test-set evidence (accuracy and supports)

From `artifacts/reports/baseline_classification_report.json` and `artifacts/reports/improved_classification_report.json`:

- Test set supports: non-fraud=42648, fraud=74
- Baseline accuracy: 0.99899349
- Improved accuracy: 0.99936801

Interpretation: Accuracy is high for both models due to extreme imbalance; this repo uses PR-AUC and tuned precision/recall as primary evidence.

5.5 Model selection (deployable choice)

From `artifacts/reports/model_selection_summary.json`: `selected_model="logistic_regression"` because improved tuned PR-AUC is lower than baseline tuned PR-AUC. The deploy metadata used by serving is stored in `artifacts/models/model_info.json`:

- `model_type="logistic_regression_pipeline"`
- `threshold=0.99`
- `n_features=30`
- `feature_columns=[Time,V1..V28,Amount]`
- `dataset_path=data/archive/creditcard.csv`

5.6 Model artifact validation checks (evidence)

From `artifacts/reports/model_validation_checks.json`:

- `model_artifact_loadable=true`
- `probabilities_in_range=true`
- `no_nan_predictions=true`
- `feature_count_matches=true`

5.7 Embedded evaluation figures

Figures [7–11](#) compare baseline and improved candidate behavior.

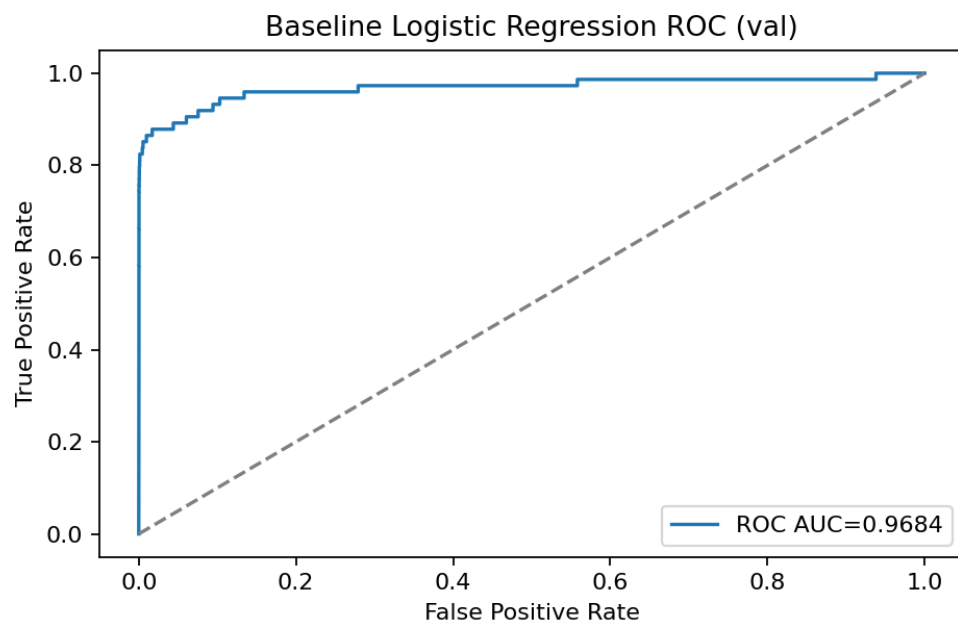


Figure 7: Baseline ROC curve. Source: `artifacts/figures/baseline_roc_curve.png`

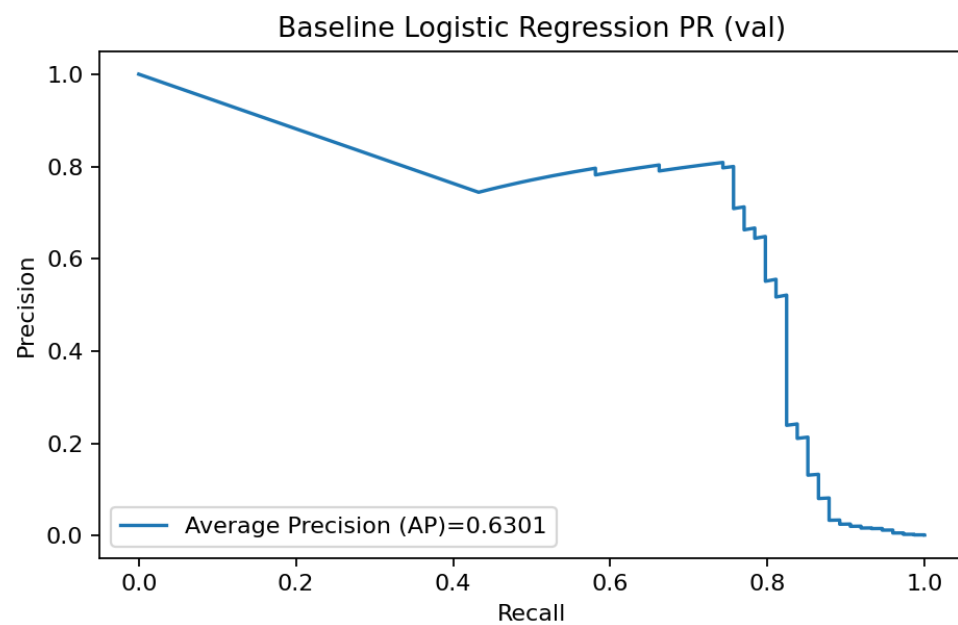


Figure 8: Baseline PR curve. Source: `artifacts/figures/baseline_pr_curve.png`

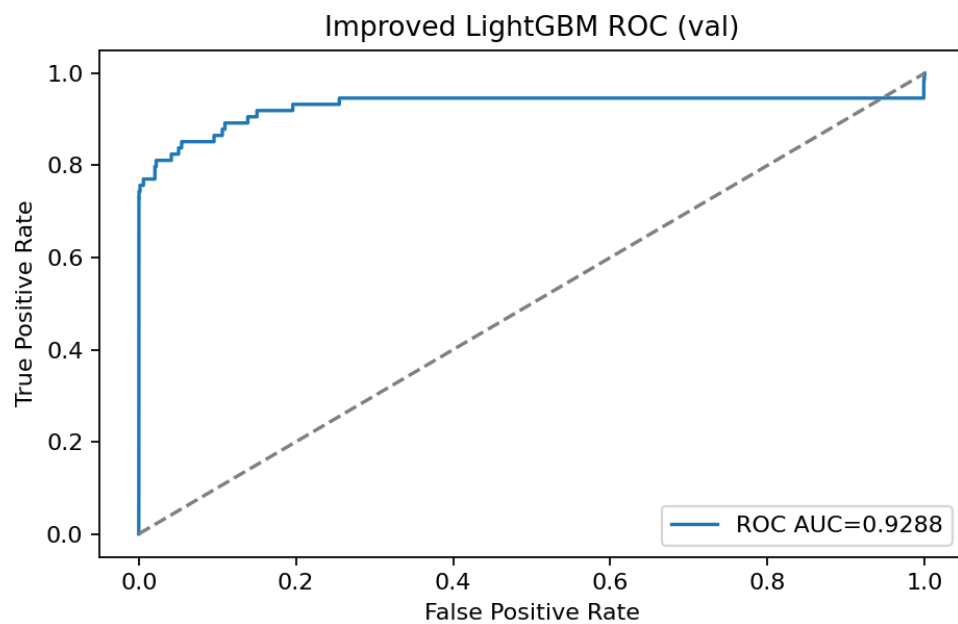


Figure 9: Improved ROC curve (LightGBM). Source: `artifacts/figures/improved_roc_curve.png`

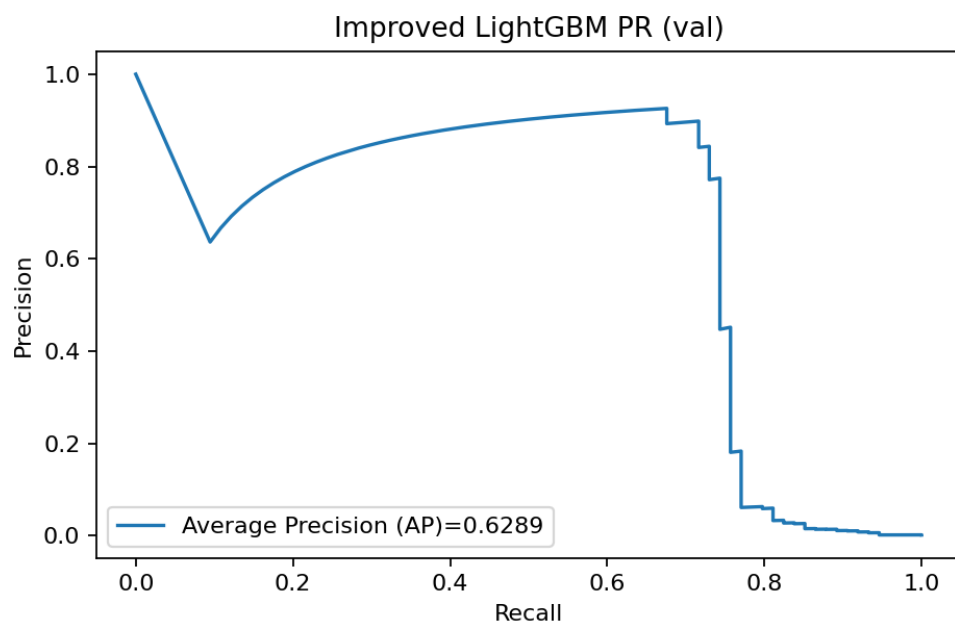


Figure 10: Improved PR curve (LightGBM). Source: `artifacts/figures/improved_pr_curve.png`

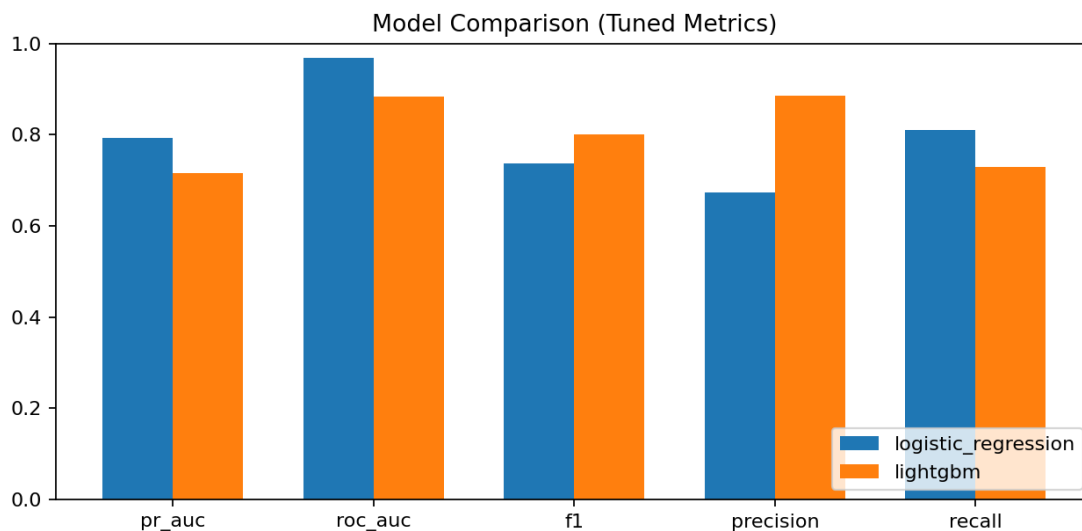


Figure 11: Model comparison (tuned metrics). Source: `artifacts/figures/model-comparison.png`

5.8 Final deployable model diagnostics

The selected deployable model diagnostics are shown in Figures 12, 13, and 14. Feature-importance evidence is provided in Figures 15 and 16 for baseline and LightGBM interpretation comparison.

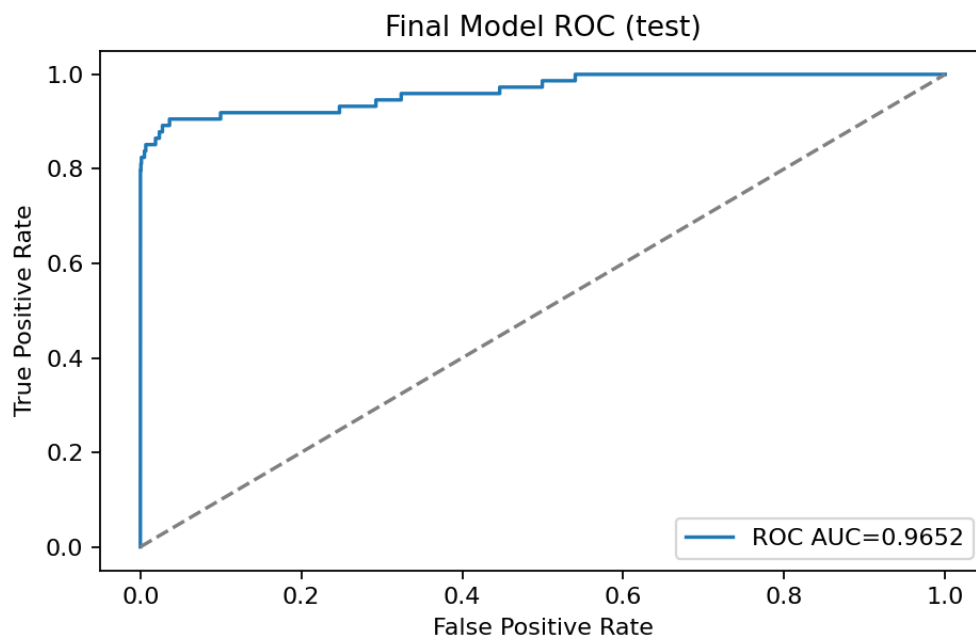


Figure 12: Final model ROC curve. Source: `artifacts/figures/final_roc_curve.png`

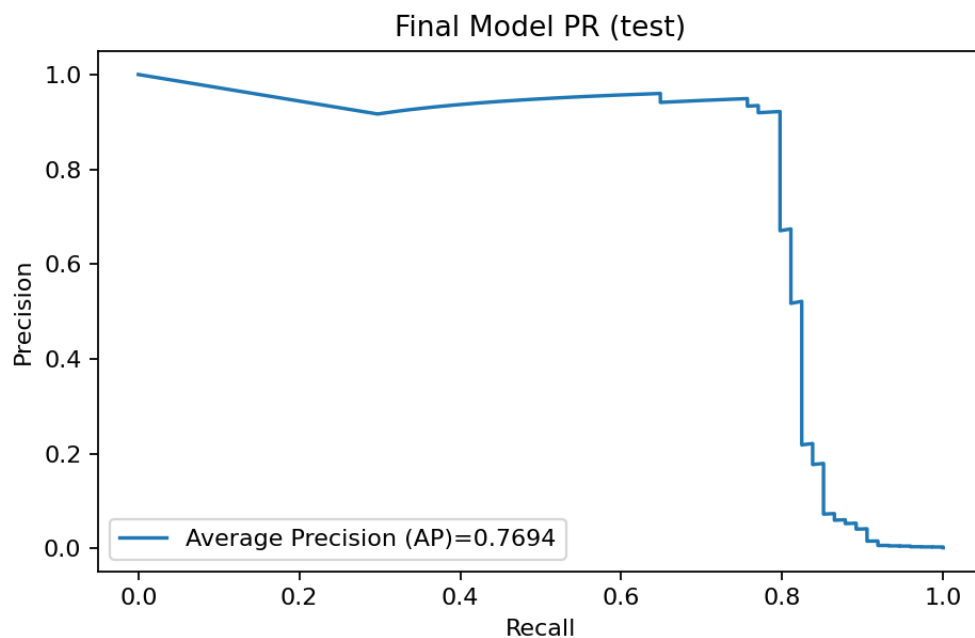


Figure 13: Final model PR curve. Source: `artifacts/figures/final_pr_curve.png`

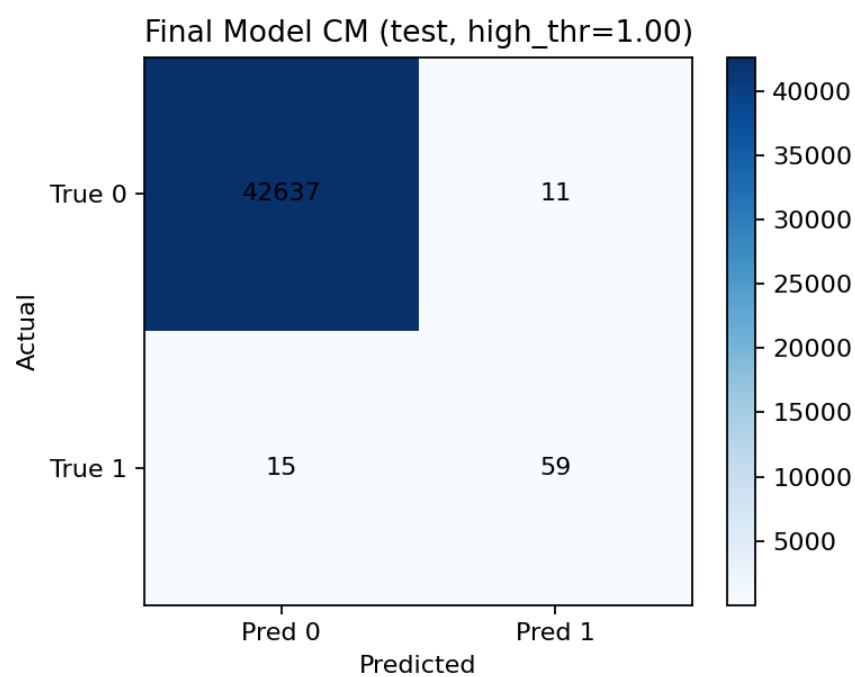


Figure 14: Final model confusion matrix. Source: `artifacts/figures/final_confusion_matrix.png`

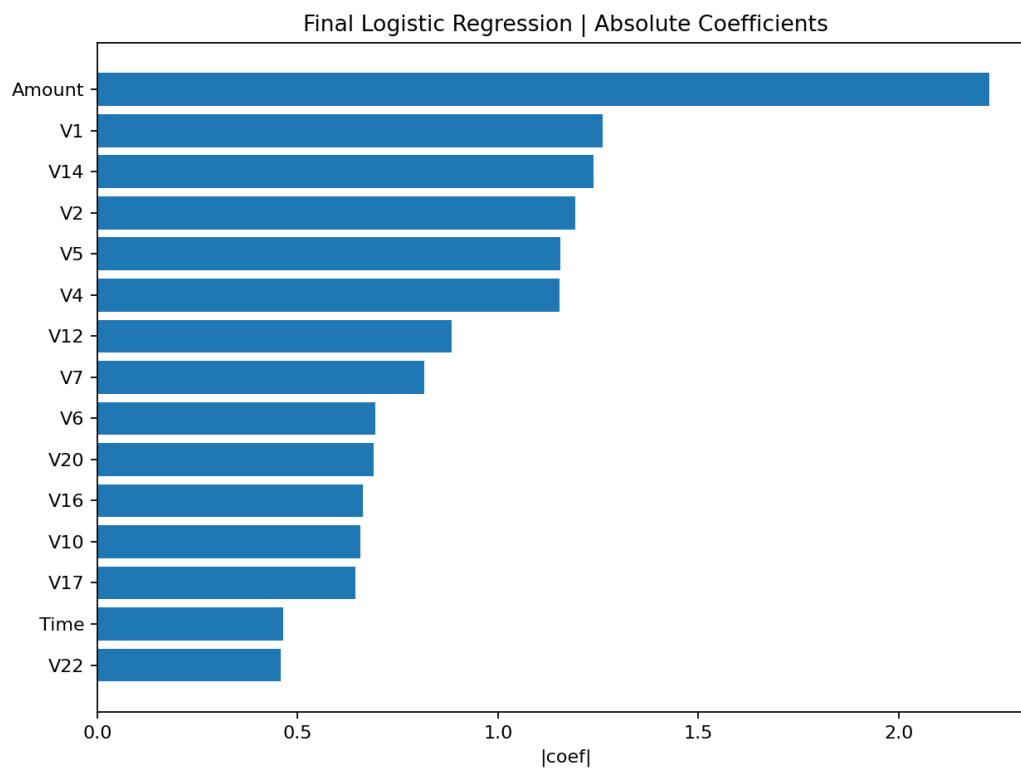


Figure 15: Final model feature importance. Source: `artifacts/figures/final_feature_importance.png`

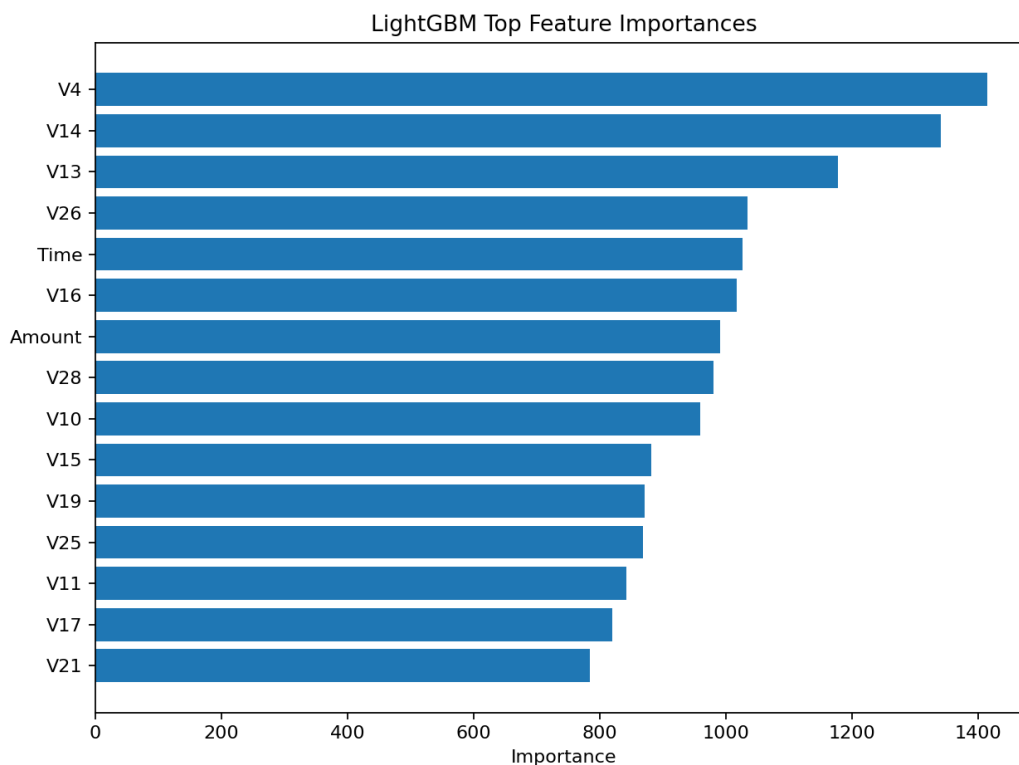


Figure 16: Improved model feature importance (LightGBM). Source: `artifacts/figures/improved_feature_importance.png`

6 Threshold Tuning

6.1 How it works (evidence)

In `src/pipelines/run_model_workflow.py`, the system:

- sweeps thresholds from 0.01 to 0.99 on validation scores
- selects the threshold that maximizes F1 on validation
- evaluates the tuned threshold on the test split and records it in model metadata

Threshold sweep tables: `artifacts/benchmarks/baseline_threshold_tuning.csv`, `artifacts/benchmarks/improved_threshold_tuning.csv`, plus overlay comparison `artifacts/benchmarks/threshold_comparison_table.csv`.

6.2 Numeric tuning evidence (from validation sweeps)

From the sweep tables (best-by-F1 on validation):

- Baseline best validation threshold: 0.99 (validation F1=0.6629)
- Improved best validation threshold: 0.84 (validation F1=0.7970)

6.3 Embedded threshold figures

Threshold behavior and cross-model operating-point differences are illustrated in Figures 17, 18, and 19.

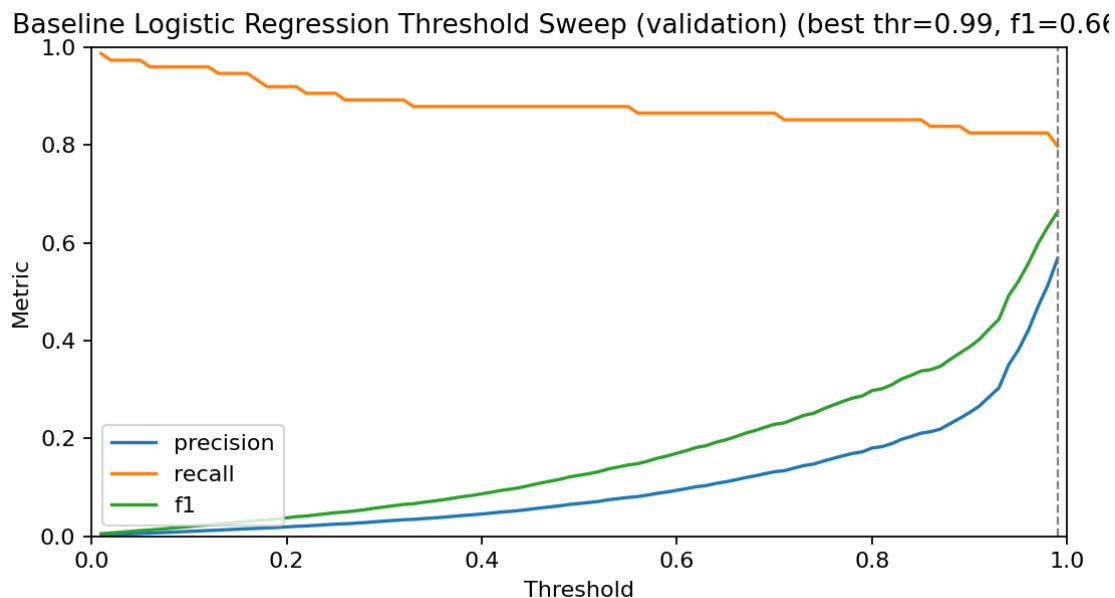


Figure 17: Baseline threshold sweep. Source: `artifacts/figures/baseline_threshold_sweep.png`

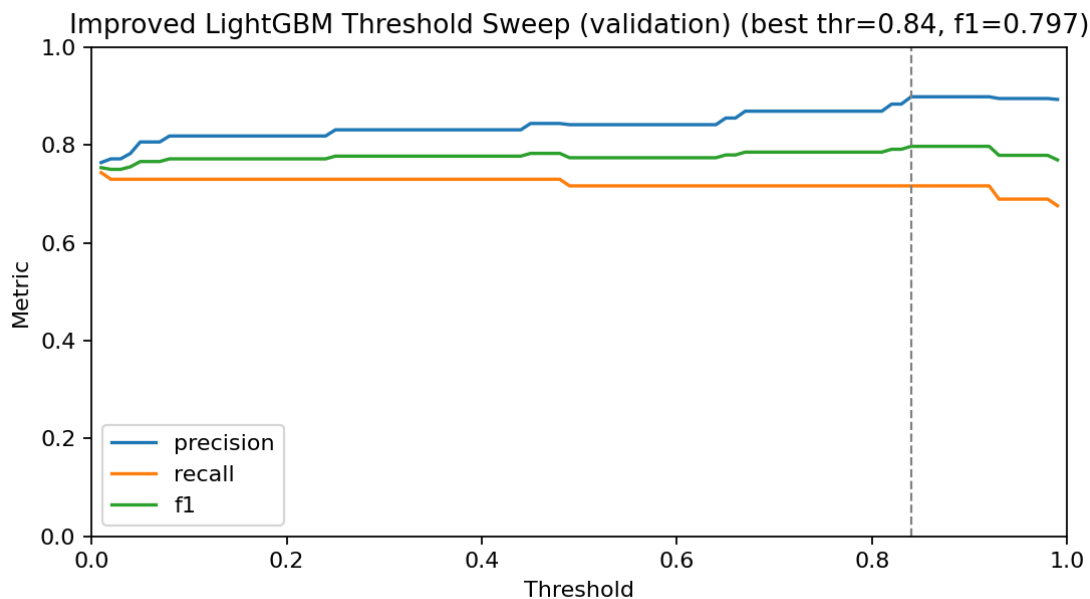


Figure 18: Improved threshold sweep. Source: `artifacts/figures/improved_threshold_sweep.png`

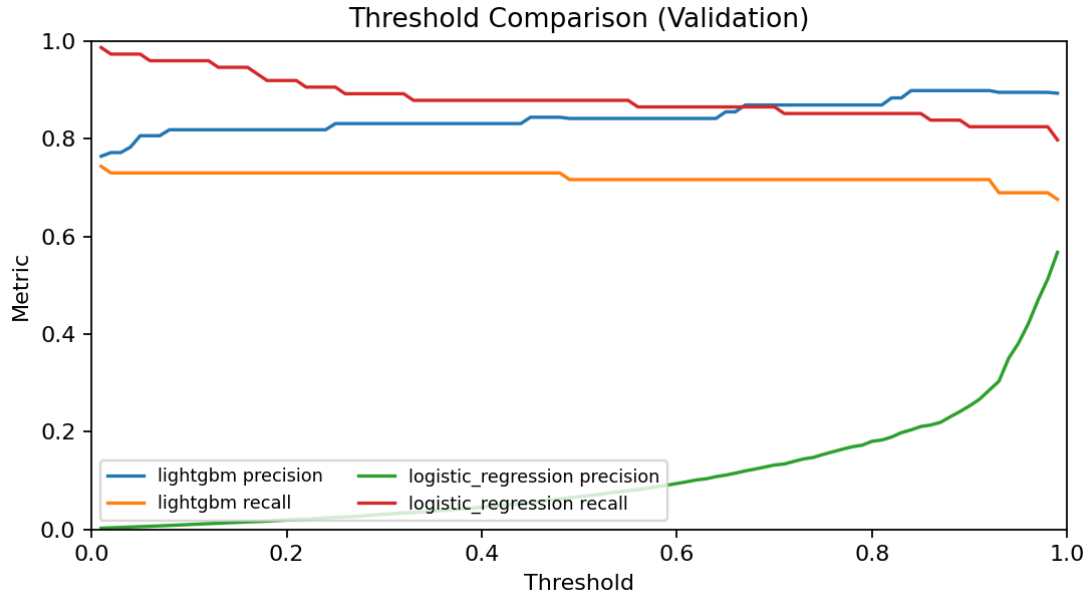


Figure 19: Threshold comparison overlay (F1 curves). Source: `artifacts/figures/threshold-comparison.png`

7 Model Explainability (SHAP)

7.1 What is implemented (evidence)

`src/pipelines/run_model_workflow.py` computes SHAP values for the LightGBM candidate model using `shap.TreeExplainer`, exporting:

- SHAP summary plot: `artifacts/figures/shap_summary.png`
- SHAP importance table: `artifacts/benchmarks/shap_importance_table.csv`

Constraint (no assumption): SHAP is generated for LightGBM even though the selected deployable model in this artifact set is Logistic Regression.

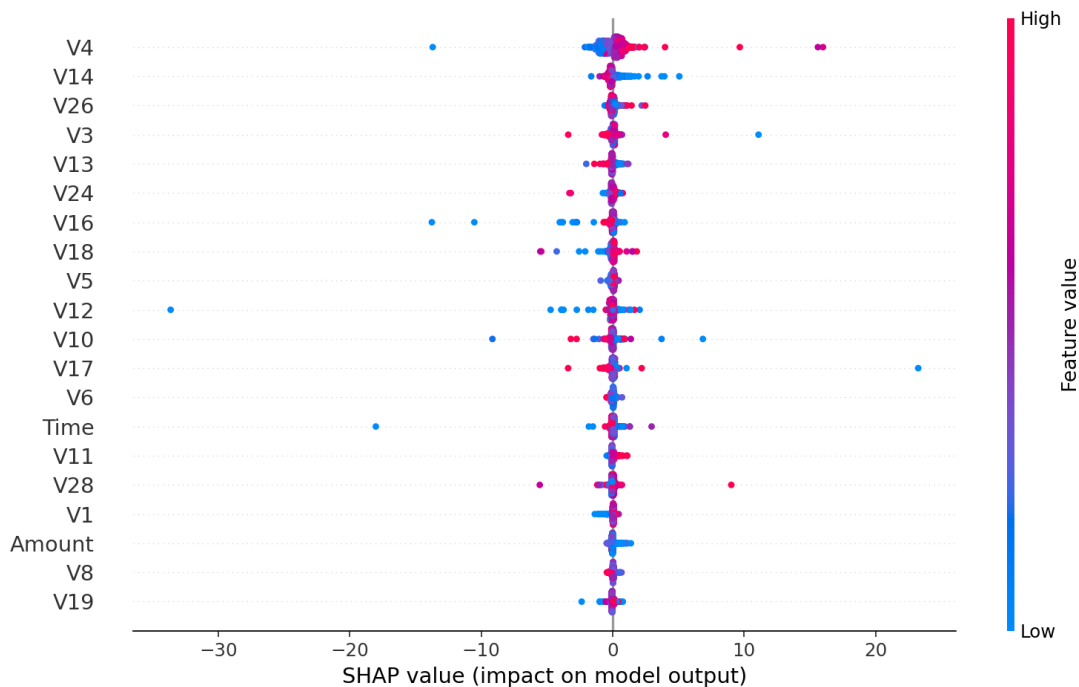


Figure 20: SHAP summary plot (LightGBM). Source: `artifacts/figures/shap_summary.png`

8 System Architecture (E2E flow)

8.1 Architecture diagram (embedded)

This diagram reflects the implemented structure described in `ARCHITECTURE.md` and wired by `deployment/docker-compose.yml`.

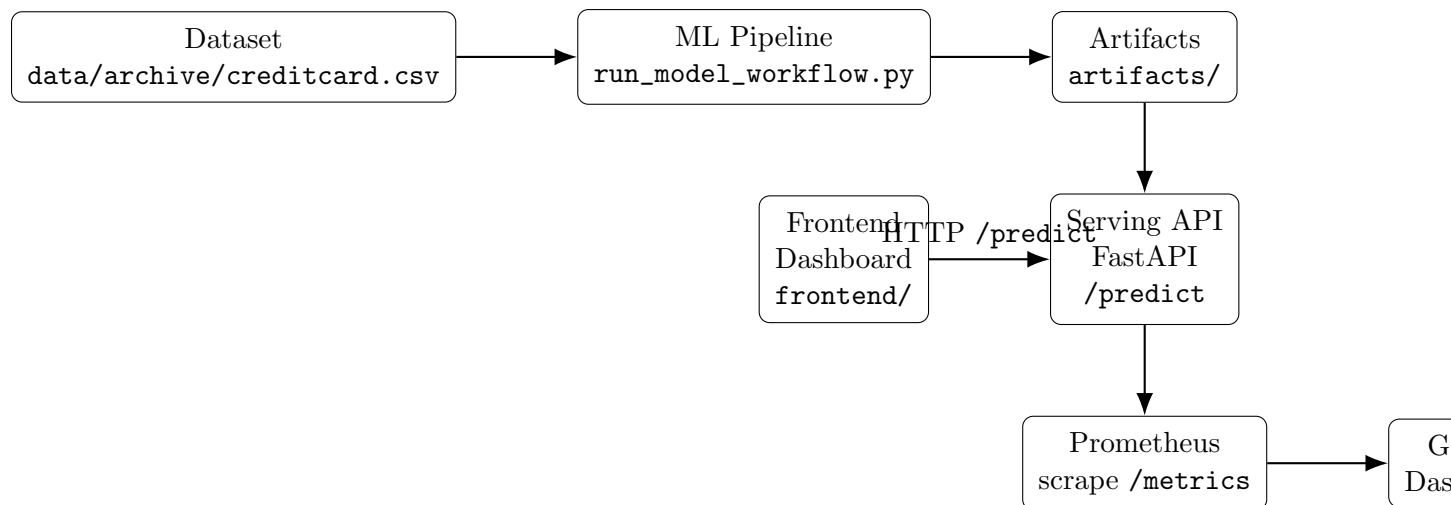


Figure 21: End-to-end architecture (implementation-aligned).

8.2 Contract boundary (evidence)

The feature contract for the Kaggle model is 30 floats ordered as [Time, V1..V28, Amount]:

- stored in `artifacts/models/model_info.json` as `feature_columns`
- enforced by API length checks in `src/api/main.py`
- enforced by frontend client checks in `frontend/api-client.js`

9 API Design

9.1 Endpoints (implemented)

Implemented in `src/api/main.py`:

- GET `/health` (model loaded status + metadata)
- POST `/predict` (fraud probability + threshold-based label)
- GET `/metrics` (Prometheus exposition)
- GET `/dataset/samples` (dataset-backed sampling; mixed/fraud/legit)
- Additional: `/features/schema`, `/features/random`

9.2 Schema evidence

Request and response shapes are defined in `src/api/schemas.py`. `/predict` returns: `request_id`, `risk_score`, `risk_tier/action`, `threshold_review/threshold_high`, `model_version`, plus optional metadata.

9.3 Runtime examples

Status: Unverified (API was not launched in this report generation environment). To capture real JSON evidence, run the repo's verification harness `verify_system.py` after starting the API, or use the command sequence in `FINAL_E2E_REPORT.md`.

10 Frontend System (UI/UX)

10.1 Dashboard layout (evidence)

From `frontend/index.html`, the dashboard includes:

- Connection pill + model metadata (version, thresholds review/high, expected features)
- Control panel (API URL, mode, speed, start/stop/reset)
- KPIs (counts, rates, avg risk score, last latency)
- Alert panel (Review/High)
- Probability-over-time chart (canvas)
- Append-only transaction feed table

10.2 Streaming and alerting logic (evidence)

From `frontend/app.js`:

- Streams transactions at 0.5s / 1s / 2s intervals, calling POST `/predict`
- Risk tiers: LOW if score < `threshold_review`; REVIEW if between thresholds; HIGH if score $\geq$ `threshold_high`
- Stops stream after 4 consecutive errors (demo stability)

Real sample pool is fetched from the backend dataset endpoint (GET `/dataset/samples`) using production-like sampling, with rare fraud spikes for demo visibility.

10.3 Embedded frontend screenshot

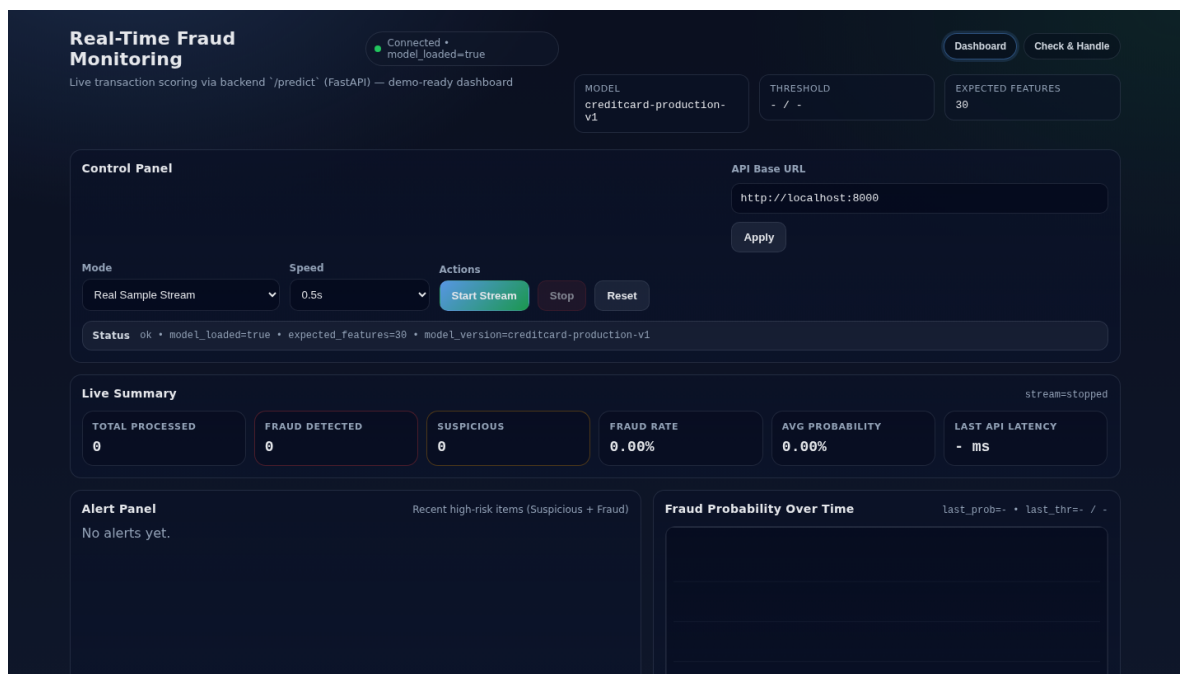


Figure 22: Frontend dashboard screenshot (connected to backend). Source: `artifacts/figures/frontend_dashboard_live.png`.

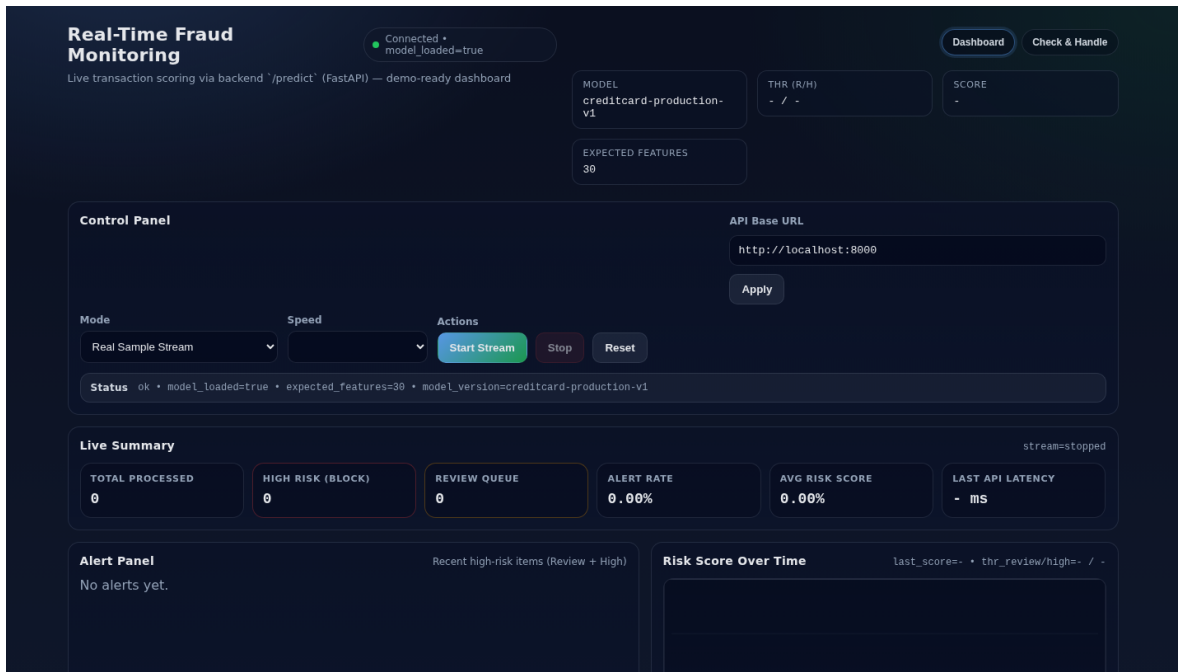


Figure 23: Frontend streaming mode screenshot. Source: `artifacts/figures/frontend-dashboard_streaming.png`.

Note: Headless screenshot capture does not reliably wait for asynchronous streaming updates; the screenshot shows a connected dashboard ready to stream. A live “stream running” screenshot is a TODO for manual capture during demo rehearsal.

11 API Behavior Examples (Captured Outputs)

11.1 How these examples were captured

To avoid network namespace issues in automated runs, API examples were captured by calling the FastAPI ASGI app directly (with lifespan startup) using the repo’s ASGI test harness (`tests/integration/asgi_client.py`). The captured outputs are saved in:

- `artifacts/reports/api_example_health.json`
- `artifacts/reports/api_example_dataset_samples.json`
- `artifacts/reports/api_example_predict.json`
- `artifacts/reports/api_example_metrics.txt`

11.2 Example: GET /health

```
{
  "status": "ok",
  "model_loaded": true,
```

```

"model_version": "creditcard-production-v1",
"expected_features": 30,
"threshold_review": 0.5,
"threshold_high": 0.99,
"model_type": "logistic_regression_pipeline",
"feature_names": [
  "Time",
  "V1",
  "V2",
  "V3",
  "V4",
  "V5",
  "V6",
  "V7",
  "V8",
  "V9",
  "V10",
  "V11",
  "V12",
  "V13",
  "V14",
  "V15",
  "V16",
  "V17",
  "V18",
  "V19",
  "V20",
  "V21",
  "V22",
  "V23",
  "V24",
  "V25",
  "V26",
  "V27",
  "V28",
  "Amount"
],
"selection_timestamp_utc": "2026-04-14T08:17:58.395658+00:00",
"score_semantics": "risk_score_uncalibrated"
}

```

11.3 Example: GET /dataset/samples?n=1&strategy=legit

```

{
  "n_features": 30,
  "feature_names": [
    "Time",
    "V1",
    "V2",
    "V3",
    "V4",
    "V5",
    "V6",
    "V7",

```



```

    "V8",
    "V9",
    "V10",
    "V11",
    "V12",
    "V13",
    "V14",
    "V15",
    "V16",
    "V17",
    "V18",
    "V19",
    "V20",
    "V21",
    "V22",
    "V23",
    "V24",
    "V25",
    "V26",
    "V27",
    "V28",
    "Amount"
],
"dataset_path":
↳  "/home/andb/Documents/Final_Project_DMM501_Group1/data/archive/creditcard.csv",
"samples": [
{
  "features": [
    163543.0,
    1.92202844531211,
    -0.209501747502902,
    -0.546351489035423,
    1.19039418874589,
    -0.139587769949728,
    0.0921754180903862,
    -0.403014714773721,
    0.0916500682424196,
    0.935213290680785,
    0.150717178853953,
    -1.23184137606726,
    0.0290278800982994,
    -0.355207330019543,
    0.102977013538,
    0.586065615634743,
    0.426050395912493,
    -0.69978408890047,
    -0.10296963786687,
    -0.365918957970945,
    -0.198400949314837,
    -0.22537970281812,
    -0.649124709614662,
    0.372838522075116,
    0.319945443545818,
    -0.365989562263036,
    -0.993610170296588,

```

```

        0.0426258164517278,
        -0.0178255131971737,
        34.8
    ],
    "class_label": 0,
    "time_s": 163543.0,
    "amount": 34.8
  }
]
}

```

11.4 Example: POST /predict

```

{
  "request_id": "904be86c-619b-44ae-8126-29dd1e965fb1",
  "risk_score": 0.015122751665770852,
  "risk_tier": "LOW",
  "action": "allow",
  "fraud_label": 0,
  "threshold_review": 0.5,
  "threshold_high": 0.99,
  "score_semantics": "risk_score_uncalibrated",
  "model_version": "creditcard-production-v1",
  "model_type": "logistic_regression_pipeline",
  "n_features": 30,
  "feature_names": [
    "Time",
    "V1",
    "V2",
    "V3",
    "V4",
    "V5",
    "V6",
    "V7",
    "V8",
    "V9",
    "V10",
    "V11",
    "V12",
    "V13",
    "V14",
    "V15",
    "V16",
    "V17",
    "V18",
    "V19",
    "V20",
    "V21",
    "V22",
    "V23",
    "V24",
    "V25",
    "V26",
    "V27",

```

```

    "V28",
    "Amount"
  ],
  "selection_timestamp_utc": "2026-04-14T08:17:58.395658+00:00"
}

```

12 Monitoring & Alerts

12.1 Metrics emitted (evidence)

From `src/monitoring/metrics.py`, the API emits:

- `api_requests_total{endpoint,method,http_status}`
- `api_request_latency_seconds` histogram (p95 computed by Prometheus)
- `fraud_predictions_total{label}`
- `fraud_scores_sum` and `fraud_scores_count` (avg score)

12.2 Prometheus scrape and rules (evidence)

Scrape config: `deployment/prometheus/prometheus.yml`. Alert rules: `deployment/prometheus/alerts.yml`, including: high 5xx error rate, high p95 latency, and prediction-rate anomaly alerts.

12.3 Grafana dashboard (evidence)

Dashboard JSON: `deployment/grafana/dashboards/fraud_api.json`.

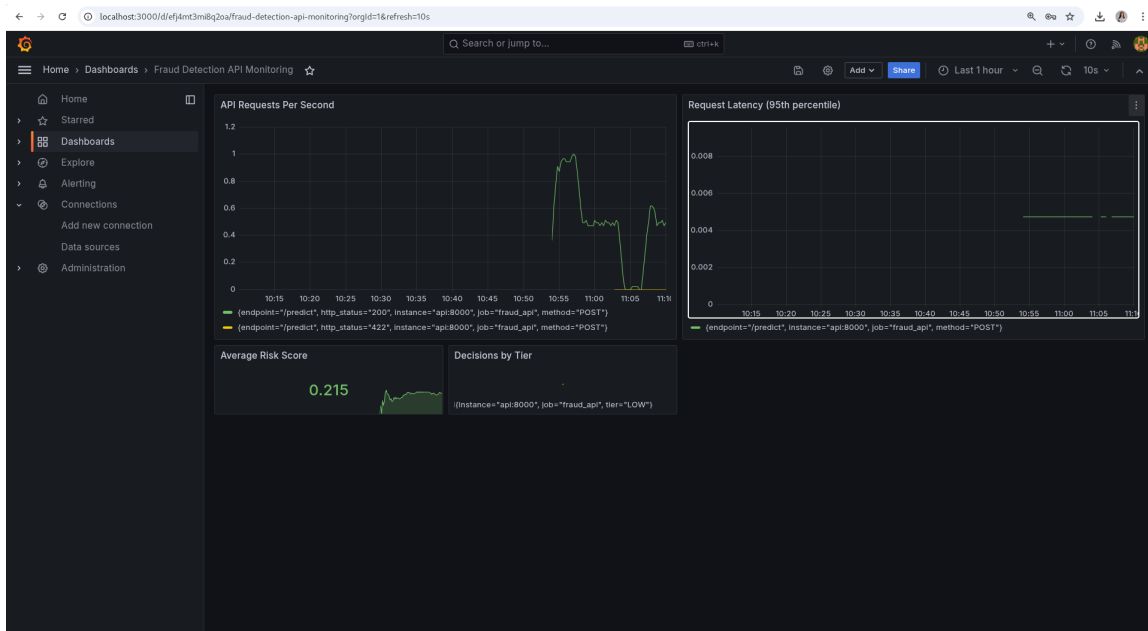


Figure 24: Grafana dashboard screenshot. Source: `artifacts/deployments/Grafana-dashboard.png`

Prometheus Alerts Graph Status Help					
Targets					
All scrape pools Unhealthy Collapse All Filter by endpoint or labels Unknown Unhealthy Healthy					
fraud_api (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://api:8000/metrics	UP	instance="api-8000" job="fraud_api"	22.67s ago	1.378ms	

Figure 25: Prometheus targets screenshot. Source: artifacts/deploy/Prometheus-targets.png

Prometheus Alerts Graph Status Help					
Rules					
fraud-api-alerts		Interval: 1m 0s		18.828s ago	0.562ms
Rule	State	Error	Last Evaluation	Evaluation Time	
alert: FraudApiHigh5xxErrorRate expr: sum(rate(api_requests_total{http_status=~"5.."}[5m])) / clamp_min(sum(rate(api_requests_total[5m]), 1)) > 0.05 for: 10m labels: severity: warning annotations: description: More than 5% of requests are 5xx over the last 10 minutes. summary: High 5xx error rate on Fraud API	OK		18.830s ago	0.323ms	
alert: FraudApiHighP95Latency expr: histogram_quantile(0.95, sum by (le) (rate(api_request_latency_seconds_bucket[5m])) > 0.5 for: 10m labels: severity: warning annotations: description: p95 request latency is above 0.5 seconds over the last 10 minutes. summary: High p95 latency on Fraud API	OK		18.830s ago	0.118ms	
alert: FraudApiPredictionRateTooLow expr: sum(rate(fraud_predictions_total[5m])) < 0.01 for: 15m labels: severity: info annotations: description: Prediction rate is below 0.01 req/s (near idle) for 15 minutes. summary: Prediction traffic is very low	OK		18.830s ago	0.046ms	
alert: FraudApiPredictionRateTooHigh expr: sum(rate(fraud_predictions_total[5m])) > 50 for: 10m labels: severity: warning annotations: description: Prediction rate is above 50 req/s for 10 minutes. summary: Prediction traffic is unusually high	OK		18.830s ago	0.034ms	

Figure 26: Prometheus alert rules screenshot. Source: artifacts/deploy/Prometheus-rules.png

13 Deployment

13.1 Docker Compose stack (evidence)

From deployment/docker-compose.yml, services include: API (8000), frontend (8080), MLflow (5000), Prometheus (9090), Grafana (3000).

```
[LightGBM] [Warning] Stopped training because there are no more leaves that meet the split requirements
/home/andb/Documents/Final_Project_DPM501_Group1/.venv/lib/python3.13/site-packages/shap/explainers/_tree.py:620: UserWarning: LightGBM binary classifier with TreeExplainer
r shap values output has changed to a list of ndarray
warnings.warn(
{
  "dataset_path": "data/archive/creditcard.csv",
  "rows": 284897,
  "features": 30,
  "threshold_review": 0.7391262534904675,
  "threshold_high": 0.9999047447184487,
  "selected_model": "logistic_regression",
  "artifacts_root": "artifacts"
}
(.venv) andb@andb1ap:~/Documents/Final_Project_DPM501_Group1$
(.venv) andb@andb1ap:~/Documents/Final_Project_DPM501_Group1$ docker compose -f deployment/docker-compose.yml up --build
[+] up 24/24
  Image prom/prometheus:v2.54.1 Pulled
  Image grafana/grafana:11.1.0 Pulled
[+] Building 9.7s (12/20)
=> [internal] load local bake definitions 0.0s
=> reading from stdin 1.54s
=> [api internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 421B 0.0s
=> [frontend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 360B 0.0s
=> [mlflow internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 125B 0.0s
=> [frontend internal] load metadata for docker.io/library/python:3.11-slim 2.7s
=> [frontend internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [mlflow internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [api internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [frontend 1/5] FROM docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf 5.2s
=> => resolve docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf 0.0s
=> => sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5f6a165 14.37MB / 14.37MB 4.5s
=> => sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf 10.37KB / 10.37KB 0.0s
=> => sha256:ad65166ad2583036804a3f772ec8f1cad733cf77d631c6840a81ea9231b6b350 1.75KB / 1.75KB 0.0s
=> => sha256:d1a5c0f8a3072e0813ee970460a8f49f2c1b0d588b17a558cc6f707084226 5.48KB / 5.48KB 0.0s
=> => sha256:5433b2dcdf5cb7fa8d5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976 29.78MB / 29.78MB 2.3s
=> => sha256:377fa6577b94e98598e2c10432eb8c66cf1a6c7098b21c613f8334606aa14 1.29MB / 1.29MB 1.8s
=> => sha256:0816ee351b60c03c7a77a3f50c717891eadd92774e6a1d59f6cce05268065606 251B / 251B 2.4s
=> => extracting sha256:5433b2dcdf5cb7fa8d5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976 1.0s
=> => extracting sha256:377fa6577b94e98598e2c10432eb8c66cf1a6c7098b21c613f8334606aa14 0.1s
=> => extracting sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5f6a165 0.6s
=> => extracting sha256:0816ee351b60c03c7a77a3f50c717891eadd92774e6a1d59f6cce05268065606 0.0s
=> [frontend internal] load build context 0.0s
=> => transferring context: 93.31KB 0.0s
=> [api internal] load build context 0.0s
=> => transferring context: 223.21KB 0.0s
=> [mlflow 2/5] WORKDIR /app 0.3s
=> [mlflow 3/3] RUN pip install --no-cache-dir mlflow 1.4s
=> [api 3/5] RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/* 1.3s
=> # Get:1 http://deb.debian.org/debian trixie InRelease
=> # Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
=> # Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
=> # Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
```

Figure 27: Docker Compose stack evidence screenshot. Source: artifacts/deploy/docker-compose.png

13.2 Deployment contract (model + threshold)

Compose sets `MODEL_PATH=/app/artifacts/models/final_model.joblib` and `FRAUD_THRESHOLD="0.99"`. The API loader reads `model_info.json` next to the model artifact to obtain `n_features`, feature names, dataset path, and selection timestamp.

14 Testing & CI/CD

14.1 Tests (evidence)

- Unit tests: `tests/unit/`
- Integration tests: `tests/integration/`
- Model/data tests: `tests/model/`, `tests/data/`

Status: **Unverified** execution in this report generation environment.

14.2 GitHub Actions workflows (evidence)

- CI + coverage gate: `.github/workflows/ci.yml` (requires coverage $\geq 80\%$)
- Docker build + compose config validation: `.github/workflows/docker.yml`

15 Responsible AI

Responsible AI documentation is provided in `RESPONSIBLE_AI.md`, including:

- fairness limitations (no protected attributes in dataset)
- explainability via SHAP (LightGBM artifacts)
- privacy stance (avoid logging raw features; prefer aggregated metrics)
- ethical risks (false positives/false negatives) and mitigation strategies

16 Limitations

16.1 Verified vs unverified in this report

- **Verified:** dataset presence; benchmark artifacts; figures; deploy metadata; frontend code; monitoring/deployment configuration files.
- **Unverified:** live API responses; running Compose stack; Grafana screenshots; executing tests and CI runs.

16.2 Multiple artifact sets (do not conflate)

This repository contains two artifact tracks:

- Kaggle benchmark track under `artifacts/models/` with 30 features (used for deployment contract and dashboard).
- Synthetic training track under `artifacts/model.joblib`, `artifacts/model_info.json`, `artifacts/metrics_report.json` with 20 features (`data_source: synthetic:make_classification`).

This report's end-to-end demo contract uses the Kaggle benchmark track (30 features).

17 Conclusion

17.1 End-to-end completeness (implementation)

At demo scope, the repository implements: real dataset, reproducible benchmark pipeline producing artifacts, deployable model + threshold policy, serving API with metrics, frontend dashboard calling the real API, and monitoring/deployment configuration.

17.2 Demo readiness (next evidence to capture)

For submission-quality proof, capture:

- real `/health`, `/predict`, `/metrics` outputs (JSON/text) while API is running
- frontend screenshot while stream is running (alerts + feed + chart)
- Grafana dashboard screenshot while streaming (optional, via Compose)

Appendix: System components status table

Component	Evidence	Status	How to verify
Dataset	<code>data/archive/creditcard.csv</code> , <code>artifacts/reports/dataset_- schema.json</code>	Verified	N/A
ML pipeline out-puts	<code>src/pipelines/run_- model_workflow.py</code> , <code>artifacts/reports/*</code> , <code>artifacts/figures/*</code>	Verified	Re-run workflow
Deployable model	<code>artifacts/models/final_- model.joblib</code> , <code>artifacts/models/model_- info.json</code>	Verified	Load via joblib (after deps)
API runtime	<code>src/api/main.py</code> + <code>tests/integration/*</code>	Unverified	Start Uvicorn + curl
Frontend runtime	<code>frontend/*</code> , screenshot <code>frontend_- dashboard.png</code>	Unverified	Serve frontend/

Component	Evidence	Status	How to verify
Monitoring run-time	deployment/prometheus/*, deployment/grafana/*	Unverified	Run Compose
CI runs	.github/workflows/*	Unverified	Check GitHub Actions